

# Day 11: Kubernetes Introduction & Setup

*Kubernetes Architecture, Single-Node Minikube Setup & Pod CLI Workflows*

---

## Module 1: Introduction to Kubernetes (K8s)

Kubernetes (often abbreviated as K8s) is an open-source platform designed to automate deploying, scaling, and managing containerized applications. Originally developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF), Kubernetes groups containers that make up an application into logical units for easy discovery and management. K8s acts as the orchestration brain, coordinating container clusters to ensure high availability and efficient resource utilization.

- **Container Orchestration:** Manages the execution lifecycles of containerized microservices across a cluster of server nodes.
- **Self-Healing:** Automatically restarts containers that fail, replaces and reschedules containers when nodes die, and kills containers that do not respond to user-defined health checks.
- **Horizontal Scaling:** Scales application instances up or down dynamically based on CPU utilization or custom metric thresholds.
- **Service Discovery & Load Balancing:** Exposes containers to networks using DNS names or unique IP addresses, automatically distributing request loads across instances.

## Module 2: Kubernetes System Architecture

A Kubernetes cluster consists of two primary operational units: the Control Plane (which manages the cluster state) and one or more Worker Nodes (which run the active container applications).

### 1. The Control Plane (Master Node components)

- **kube-apiserver:** The front-end entry point for the Kubernetes control plane. It exposes the Kubernetes API and handles all administrative commands (e.g. from kubectl tools).
- **etcd:** A consistent and highly-available key-value store used as Kubernetes' backing database for all cluster state data.
- **kube-scheduler:** Monitors newly created Pods that have no assigned node, and selects the healthiest worker node for them to run on based on resource capacities.
- **kube-controller-manager:** Runs controller processes that regulate the state of the cluster (Node Controller, Job Controller, Endpoint Controller).

## Module 2: Kubernetes Architecture (Continued)

### 2. Worker Node Components

- **kubelet:** An agent that runs on each worker node in the cluster. It ensures that the containers described in Pod specifications are running and healthy.
- **kube-proxy:** A network proxy running on each node that maintains network rules, allowing communication to Pods from inside or outside the cluster.
- **Container Runtime:** The software responsible for running containers (e.g., containerd, Docker Engine).

### 3. Core Abstraction: The Pod Object

A Pod is the smallest and simplest deployable unit in the Kubernetes object model. A Pod represents a single instance of a running process in your cluster. It wraps one or more tightly coupled application containers, storage resources, a unique network IP, and local port configurations. Containers inside a single Pod share the same network namespace (meaning they can communicate via localhost) and storage volumes.

## Module 3: Local Kubernetes Setup & Deployment Walkthrough

This section maps and explains the commands executed in your terminal history to set up a local Kubernetes cluster using Minikube, build a local Docker image inside Minikube's engine, deploy a Pod, and expose it publicly.

### Day 11 Terminal Command Sequence Walkthrough

- **1. System Update & Docker Installation:** Updates OS packages and installs Docker to act as the container driver container runtime:  
`sudo apt update && sudo apt install docker.io -y`
- **2. Download and Install kubectl CLI:** Downloads the latest stable release binary of the Kubernetes CLI and places it in the system path:  
`curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"`  
`chmod +x kubectl && mv kubectl /usr/local/bin/`
- **3. Download and Run Minikube:** Downloads Minikube (local single-node cluster tool) and launches it using Docker as the driver runtime:  
`curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64`  
`minikube start --driver=docker --force`

## Module 3: Setup & Deployment Walkthrough (Continued)

- **4. Cluster Verification:** Checks cluster info and node states to verify Minikube is active:

```
kubectl cluster-info
```

```
kubectl get nodes
```

- **5. Point Terminal to Minikube Docker Daemon:** Points your terminal's docker CLI directly to the docker daemon running inside the Minikube virtual machine. This allows images to be built directly inside the cluster's filesystem without external registry pushes:

```
eval $(minikube docker-env)
```

- **6. Build Application Container Image:** Builds the application container image directly within the Minikube environment:

```
docker build -t ubuntu-hospital-web .
```

- **7. Deploy the Pod:** Instructs Kubernetes to create the Pod described in your configuration file:

```
kubectl apply -f pod.yml
```

- **8. View Logs:** Verifies container launching and checks output log streams:

```
kubectl logs hospital-pod
```

- **9. Expose Pod to Public Traffic (Port Forwarding):** Forwards traffic from port 5000 on the host system to port 5000 inside the running pod. The '--address=0.0.0.0' binds it to all local network interfaces, making the CarePlus web portal accessible via your EC2 public IP address:

```
kubectl port-forward hospital-pod 5000:5000 --address=0.0.0.0
```

### Deployment File Blueprint: pod.yml

Below is the YAML specification used to define the Kubernetes Pod object running the CarePlus Hospital web application.

```
apiVersion: v1
kind: Pod
metadata:
  name: hospital-pod
  labels:
    app: hospital
spec:
  containers:
    - name: hospital-container
      image: ubuntu-hospital-web
      imagePullPolicy: Never
      ports:
        - containerPort: 5000
```